TIFC2DEV-JAVA-MJ5 · Laboratorio de Práctica

Sistema de Clínica Veterinaria

Herencia · Clases Abstractas · Interfaces

¿Qué vas a construir? Un sistema que modele los animales de una clínica veterinaria aplicando herencia, clases abstractas e interfaces de forma integrada. No hay código de inicio: tú decides la estructura. Cada decisión debe poder justificarse en voz alta.

Dominio	Clínica veterinaria · atención y seguros
Aplica	Herencia, clase abstracta, interfaces, polimorfismo
Entregable	Proyecto IntelliJ con todas las clases + Main funcional
Defensa	Debes poder explicar cada decisión de diseño en voz alta
Restricción	Sin **casting** ni **instanceof**: se resuelve solo con lo visto en clase

El sistema

La clínica **VetCare** atiende tres tipos de animales: perros, gatos y aves. Cada animal tiene nombre, edad y el nombre de su dueño. Todos tienen una consulta con costo, pero ese costo se calcula distinto según la especie.

Además, la clínica necesita registrar qué animales están **vacunados** y cuáles son **asegurables**, sin que eso dependa únicamente de ser un animal. También debe imprimir una ficha de presentación de cualquier animal con sus datos y el costo de su consulta, sin saber de antemano qué especie es.

1 · Las piezas del sistema

Antes de escribir una sola línea, analiza qué clases necesitas, cómo se relacionan y por qué cada una existe. Usa las preguntas como guía de razonamiento.

La clase base: Animal

Existe como concepto, nunca como objeto concreto. Define lo que todos los animales tienen en común y exige que cada especie resuelva lo que solo ella sabe.

Tu tarea — diseñar Animal

A. Atributos comunes. Identifica los tres datos que todo animal tiene. Decide qué modificador de acceso corresponde para que las subclases puedan usarlos directamente, y justifica por qué no son *private* ni *public*.

B. El método que todas deben resolver. La clínica calcula el costo de la consulta. ¿Por qué ese método no puede implementarse en Animal? Decláralo *abstract*, con la firma y el tipo de retorno correctos.

C. El método que todas heredan igual. La ficha muestra siempre los mismos datos en el mismo formato. Impleméntalo en Animal para que todas las subclases lo hereden sin reescribirlo. Debe llamar internamente al método del punto B.

Para pensar

¿Qué pasaría si alguien intenta hacer **new Animal("Luna", 3, "Carlos")**? ¿Java lo detecta en compilación o en ejecución? ¿Por qué?

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados. · linkedin.com/in/ana-alvarado-instructora-full-stack

Las subclases concretas: Perro · Gato · Ave

Cada especie extiende Animal y resuelve lo que solo ella sabe: cómo calcular el costo de su consulta.

Especie	Costo de consulta	Atributo propio
Perro	$45.000 base + $3.000 por cada año de edad	raza (String)
Gato	$38.000 fijo, sin importar la edad	esEsterilizado (boolean)
Ave	$28.000 base + $100 por cada gramo de peso	pesoGramos (double)

Antes de escribir cada subclase, responde

- ¿Qué atributo propio necesita además de los heredados?
- ¿Cómo llamas al constructor del padre desde la subclase?
- ¿Qué anotación indica que estás cumpliendo un método del padre?
- ¿Por qué Java te obliga a implementar el método de costo de consulta?

2 · Los contratos de capacidad

La herencia modela lo que un objeto **es**. Pero algunas capacidades cruzan jerarquías: aplican a objetos que no tienen relación entre sí. Para eso existen las interfaces.

Interfaz Vacunable

La clínica lleva registro de vacunación. Los perros y gatos se vacunan; las aves tienen un protocolo distinto que por ahora no se implementa. Vacunable declara dos capacidades:

Método	Qué representa	Retorna
registrarVacuna(String nombre)	Registra el nombre de una vacuna aplicada	void
getVacunasAplicadas()	Cuántas vacunas lleva el animal	int

Para pensar

¿Por qué esta capacidad va en una interfaz y no como método abstracto en Animal? Piensa qué pasaría si mañana la clínica también vacuna peces o reptiles, que no son ni perros ni gatos.

Interfaz Asegurable

La clínica ofrece planes de seguro. Los perros y gatos pueden asegurarse. También puede asegurarse la **Clínica** como institución, aunque la clínica no es un animal.

Método	Qué representa	Retorna
calcularPrimaSeguro()	Prima anual según las reglas de cada clase	double
obtenerNumeroPoliza()	Identificador único de la póliza	String

Reglas de negocio para la prima:

Perro	$80.000 anuales por cada año de edad

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados. · linkedin.com/in/ana-alvarado-instructora-full-stack

Gato	$120.000 fijo si está esterilizado, $200.000 si no
Clínica	$5.000.000 fijos por año

Para pensar

El momento de quiebre. Cuando implementes Clinica, nota que no extiende Animal. Aun así puede implementar Asegurable. ¿Por qué es posible? ¿Qué dice eso sobre la diferencia entre herencia e interfaces?

3 · La clase Clinica

Clinica no extiende nada. Tiene un nombre, una dirección y puede asegurarse. Su propósito aquí es demostrar que Asegurable no depende de la jerarquía de animales.

Tu tarea — construir Clinica

Define los dos atributos, el constructor y los dos métodos de Asegurable. El número de póliza puede tener el formato **POL-CLINICA-[NOMBRE]**. Sin más complejidad.

4 · El mapa del sistema

Antes de codificar, completa este mapa. Para cada clase decide qué extiende, qué implementa y por qué.

Clase	¿Extiende?	¿Implementa?	¿Abstracta?
Animal	—	—	Sí
Perro	_____	_____	No
Gato	_____	_____	No
Ave	_____	_____	No
Clinica	_____	_____	No

5 · Las pruebas en el Main

El main no prueba que el código compila: prueba que el diseño tiene sentido. Construye la secuencia en este orden y verifica la salida en consola después de cada bloque.

1 · Fichas con referencia polimórfica

Crea un arreglo de tipo **Animal[]** con un perro, un gato y un ave, cada uno instanciado con su subclase concreta. Recórrelo con un ciclo y llama **imprimirFicha()** sobre cada uno.

La referencia es abstracta (Animal), el objeto es concreto. Eso es polimorfismo.

Para pensar

El método imprimirFicha() vive en Animal. ¿Cómo sabe Java cuál costo de consulta mostrar para cada especie, si el método está en la clase padre?

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados. · linkedin.com/in/ana-alvarado-instructora-full-stack

2 · Vacunar los que aplican

Solo el perro y el gato implementan Vacunable. Declara el perro y el gato con su **tipo concreto** (Perro y Gato, no Animal), para poder llamar **registrarVacuna(...)** directamente. Registra dos vacunas en el perro y una en el gato. Luego imprime cuántas lleva cada uno.

Para pensar

Si declaras el perro como **Perro** puedes vacunarlo sin problema. ¿Por qué el ave no podría llamar registrarVacuna() aunque quisieras? ¿Qué te dice el compilador si lo intentas? Pruébalo y lee el mensaje.

3 · Calcular las pólizas

Crea una instancia de Clinica. Ahora tienes tres objetos asegurables: el perro, el gato y la clínica. Mételos en un arreglo de tipo **Asegurable[]** y recórrelo con un ciclo. Para cada uno imprime el número de póliza y la prima anual.

Fíjate: no necesitas ningún cast. Las tres clases implementan Asegurable, así que caben en el arreglo del tipo del contrato de forma natural.

Para pensar

¿Por qué el perro cabe tanto en un arreglo **Animal[]** como en uno **Asegurable[]**? ¿Qué representa cada uno de esos dos tipos para el mismo objeto?

4 · La verificación final

Sin modificar ninguna clase que ya construiste, crea una clase nueva **Conejo** que extienda Animal con un costo de consulta de $35.000 fijos. Agrégalo al arreglo Animal[] del primer bloque.

Cuenta: ¿cuántas líneas tuviste que cambiar fuera de la clase nueva? Eso mide si tu diseño es realmente extensible.

6 · ¿Cómo saber si está bien?

Tu sistema está completo cuando puedes responder **sí** a cada punto sin leer el código, solo razonando.

Diseño

- No es posible hacer new Animal(...): el compilador lo impide.
- Perro, Gato y Ave no compilan sin implementar el costo de consulta.
- Clinica no extiende Animal y aun así puede asegurarse.
- Ave no implementa Vacunable, y eso no genera ningún error.

Polimorfismo

- imprimirFicha() da salida distinta por especie sin preguntar el tipo.
- Un arreglo Animal[] contiene perros, gatos y aves mezclados.
- Un arreglo Asegurable[] contiene perros, gatos y la clínica mezclados.

Extensibilidad

- Agregar Conejo solo requirió crear una clase nueva: ninguna existente cambió.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados. · linkedin.com/in/ana-alvarado-instructora-full-stack

Para pensar

Recuerda: el objetivo no es que funcione, es que puedas justificar cada decisión. ¿Por qué abstract aquí? ¿Por qué interface allá? ¿Por qué esta clase extiende y no implementa? Si tienes la respuesta sin dudar, el ejercicio está bien hecho.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados. · linkedin.com/in/ana-alvarado-instructora-full-stack